

HTB ACADEMY

Intro to Network Traffic Analysis

HTB Academy · Network Traffic Analysis Track · Study Notes

A complete illustrated walkthrough of traffic capture and analysis with TCPDump, Wireshark, and TShark — protocol layering, filter syntax, file extraction from FTP and HTTP streams, and decrypting a captured RDP session. Lab answers are partially masked as a hint.

Platform	HackTheBox Academy
Module	Intro to Network Traffic Analysis
Completion	100% — all 15 sections
Analyst	Chayan Panchal · @tron874
Core Skills	TCPDump, Wireshark, TShark, protocol analysis, file carving, RDP decryption

0 Quick Reference — Read This First



TCPDump / Wireshark quick command index

```
tcpdump -i eth0 -nn -XX -c 100 -w out.pcap      # capture cheat-sheet
tcpdump host X and tcp port 443 / not icmp      # filter cheat-sheet
ip.addr==X / tcp.port==443 / ftp-data / tcp.stream eq 0 # Wireshark filters
File -> Export Objects -> HTTP                  # extract embedded files
Follow -> TCP Stream -> Show as Raw             # reconstruct a transferred file
```

1 What Is Network Traffic Analysis?

NTA is the practice of examining network traffic to characterize ports/protocols, establish a baseline, monitor and respond to threats, and maximize visibility across the network.

Skill	Why it matters
TCP/IP stack & OSI model	Understand how traffic and host applications interact
Common ports & protocols	Quickly identify standard vs. abnormal traffic
IP packets & sublayers	TCP (stream) vs UDP (fire-and-forget) behaviour
Protocol encapsulation	Read headers as PDUs are wrapped layer by layer

NOTE Key use cases covered: setting a day-to-day traffic baseline, identifying traffic from suspicious hosts or non-standard ports, detecting malware on the wire (ransomware, exploits, C2 callbacks), and supporting both proactive threat hunting and after-the-fact incident investigation.

2 TCPDump — Capture & Filtering



TCPDump — capture basics

```
sudo tcpdump -D                # list interfaces
sudo tcpdump -i eth0 -nn       # capture, no hostname/port resolution
sudo tcpdump -i eth0 -XX       # + ethernet header, hex + ASCII
sudo tcpdump -i eth0 -c 100    # stop after 100 packets
sudo tcpdump -i eth0 -w out.pcap # save capture to file
sudo tcpdump -r out.pcap       # read capture from file
```



TCPDump — packet filters

```
sudo tcpdump -i eth0 host 172.16.146.2    # to/from a host
sudo tcpdump -i eth0 src host 172.16.146.2 # FROM a host only
sudo tcpdump -i eth0 tcp port 443         # specific TCP port
sudo tcpdump -i eth0 portrange 0-1024     # port range
sudo tcpdump -i eth0 host X and port 23   # combined filter
sudo tcpdump -r sus.pcap icmp or host X   # OR condition
sudo tcpdump -r sus.pcap not icmp         # exclude a protocol
sudo tcpdump -i eth0 'tcp[13] &2 != 0'    # TCP SYN flag (scan detection)
```

3 Wireshark & TShark — Display Filters



Wireshark — display filters

```
ip.addr == 172.16.146.2      # to/from a specific IP
ip.src == 172.16.146.2      # FROM a specific IP only
tcp.port == 443              # filter by TCP port
tcp.port != 80               # exclude a port
dns / http / ftp / arp      # filter by protocol
ftp.request.command          # FTP command channel only
ftp-data                     # FTP data channel only
tcp.stream eq 0              # follow a specific TCP stream
tcp.port == 3389 / rdp       # RDP traffic (by port / after decrypt)
```



TShark — CLI equivalents

```
tshark -D                    # list capture interfaces
sudo tshark -i eth0 -w /tmp/test.pcap # capture, save to file
sudo tshark -i eth0 -f "host 172.16.146.2" # capture with a host filter
```

4 Lab Walkthrough — File Extraction & Live Capture



HTTP File Extraction

1. OPEN THE CAPTURE

Wireshark-lab-2.pcap, filtered for HTTP traffic

2. FIND THE IMAGE

Filter: http && image-jfif -> locates JPEG/JFIF objects

3. EXTRACT IT

File -> Export Objects -> HTTP -> save the image



FTP File Extraction

1. FILTER COMMANDS

ftp.request.command -> see what files were transferred

2. SWITCH TO DATA CHANNEL

Filter: ftp-data -> follow the TCP stream

3. SAVE AS RAW

Follow TCP Stream -> Show as Raw -> reconstructs the original file



Live Capture — RDP Session

1. CONNECT

xfreerdp to the lab target over the VPN

2. CAPTURE LIVE

Wireshark running on interface ENS224 during the session

3. FOLLOW HTTP

Followed the HTTP stream to identify the suspicious user

NOTE The live-capture PCAP couldn't be exported from the RDP lab environment — analysis had to happen directly inside the session, which is closer to how a real live-response engagement often works.



Lab questions & answers (masked)

Q1 — Filename of the image evidence found via FTP extraction -> R.....U.....g

Q2 — Employee suspected of malicious activity (live capture) -> bob

5 Decrypting RDP Traffic with a Recovered Key

The module's final section demonstrates decrypting a captured RDP session in Wireshark once the server's private key has been recovered — Edit -> Preferences -> Protocols -> TLS -> add the RSA key list entry (IP, port 3389, key file), after which the `rdp` filter starts showing decrypted application data instead of opaque TLS records.

ABOUT THIS FILE `server.key` in this folder is the lab's own teaching key, provided by HTB specifically for this decryption exercise — it decrypts a sample capture from a lab VM that no longer exists, not any live infrastructure. It's included here as one of the lab's original source materials.